Comprendre les commandes Git et les trois zones du dépôt

Ce support contient des exercices corrigés pour comprendre les commandes Git, leur utilité et leur effet sur les trois zones principales : Working Directory, Staging Area et HEAD.

1. Comprendre les trois zones de Git



Exercices Corrigés :

Exercice 1

1. Vérifier la version de Git.
2. Configurer le nom de l'utilisateur.
3. Configurer l'adresse e-mail.
4. Afficher la configuration.
5. Définir la branche initiale par défaut sur main.

Solution

```
git --version
git config --global user.name "Votre Nom"
git config --global user.email "votre.email@example.com"
git config --list
git config --global init.defaultBranch main
```

Exercice 2

Créer un nouveau dépôt Git, vérifier son état et comprendre la différence entre git init et git clone.

Solution :

1. Créer un dépôt local :

```
mkdir BookStore
cd BookStore
git init
git status
```

2. Copier un dépôt distant

```
git clone https://example.com/BookStore.git
cd BookStore
 git status
```

3. Différence :
    a. git init initialise un dépôt Git dans le dossier courant.
    b. git clone récupère un dépôt distant, son historique et ses fichiers.

Exercice 3

Dans un dépôt existant :

6. Modifier src/App.jsx.
7. Examiner l'état du dépôt.
8. Afficher les modifications.
9. Ajouter le fichier à l'index.

10. Examiner les modifications indexées.
11. Créer un commit.
12. Afficher l'historique.

Exercice 4 :

1. Crée une application React avec Vite nommée book-manager.
2. Installe les dépendances du projet.
3. Installe Redux et React-Redux.
4. Lance le serveur de développement.
5. Initialise le dépôt Git si nécessaire.
6. Vérifie l'état du dépôt.

Commandes à utiliser : npm create vite@latest , npm install, npm run dev, git init, git status.

Exercice 3

1. Ajoute les fichiers du projet à la zone de préparation.
2. Crée un commit décrivant l'initialisation du projet.
3. Affiche le dernier commit.

Commandes à travailler : git add ., git commit -m, git log -1 --oneline.

Exercice 4

1. Affiche l'état du dépôt.
2. Affiche les différences avant la préparation.
3. Prépare uniquement src/App.jsx.
4. Affiche les différences indexées.
5. Crée un commit clair.

6. Affiche l'historique compact.

Commandes à travailler : git status, git diff, git add, git diff --staged, git commit, git log --oneline.

Exercice 6

1. Explique pourquoi il est généralement inutile de versionner node_modules.
2. Crée un fichier .gitignore adapté à React et Vite.
3. Ajoute des règles pour ignorer dist, les fichiers .env et les fichiers locaux appropriés.
4. Vérifie que Git ignore les fichiers prévus.
5. Retire un fichier .env déjà suivi de l'index sans supprimer sa copie locale.
6. Crée un commit qui enregistre les règles et le retrait du suivi.

Commandes à travailler : .gitignore, git status, git add, git rm --cached, git commit.

Partie 2 — Développer avec Redux

Exercice 7

1. Crée un fichier booksSlice.js pour gérer les livres.
2. Ajoute l'action likeBook pour marquer un livre comme aimé.
3. Ajoute l'action unlikeBook pour retirer ce statut.
4. Ajoute l'action addToReadList pour placer un livre dans la liste « À lire » sans doublon.
5. Ajoute l'action removeFromReadList pour retirer un livre de cette liste.
6. Vérifie que les actions modifient correctement l'état Redux.

Exercice 8 — Connecter Redux à React

1. Configure le store Redux.
2. Connecte l'application avec Provider.
3. Utilise useSelector pour lire les livres et la liste « À lire ».
4. Utilise useDispatch pour déclencher les actions.
5. Teste les boutons « Liker », « Ne plus aimer », « Ajouter à À lire » et « Retirer de À lire ».

Exercice 9 — Préparer la branche d'intégration

1. Crée la branche develop à partir de la version actuelle.
2. Affiche les branches locales.
3. Affiche les branches locales et les références distantes connues.
4. Crée la branche feature/like-book.
5. Bascule de feature/like-book vers develop.
6. Supprime la branche de fonctionnalité après sa fusion, si elle n'est plus nécessaire.

Commandes à travailler : git branch, git branch -a, git switch, git switch -c, git branch -d.

Exercice 10 — Développer une fonctionnalité complète

1. Crée feature/add-to-read-list à partir de develop.
2. Modifie le reducer et l'interface React.
3. Vérifie les changements avec git status et git diff.
4. Prépare uniquement les fichiers concernés.
5. Vérifie le contenu indexé.
6. Crée un commit avec un message explicite.
7. Reviens sur develop et fusionne la fonctionnalité.

Commandes à travailler : git switch, git status, git diff, git add, git diff --staged, git commit, git merge --no-ff.

Partie 3 — Fusionner et résoudre les conflits

Exercice 11 — Simuler un conflit

1. Crée deux branches à partir d'un même commit.
2. Modifie la même ligne d'un fichier dans les deux branches.
3. Crée un commit dans chaque branche.
4. Fusionne les branches et observe le conflit.
5. Utilise git status pour identifier les fichiers concernés.
6. Résous le conflit dans le fichier.

7. Ajoute le fichier résolu à l'index.
8. Termine la fusion et vérifie l'historique.

Commandes à travailler : git merge --no-ff, git status, git add, git commit, git log --graph.

Partie 4 — Rebase et récupération de changements

Exercice 12 — Réorganiser l'historique avec rebase

1. Crée une branche de fonctionnalité depuis develop.
2. Crée au moins deux commits sur cette branche.
3. Ajoute un nouveau commit sur develop.
4. Rebase la branche de fonctionnalité sur develop.
5. Si un conflit survient, résous-le et poursuis le rebase.
6. Annule un rebase en cours pour revenir à son état initial.

Commandes à travailler : git rebase develop, git status, git add, git rebase --continue, git rebase --abort.

Partie 5 — Annuler ou récupérer des modifications

Exercice 13 — Choisir la bonne méthode d'annulation

1. Tu as modifié src/App.jsx sans utiliser git add. Annule ces modifications.
2. Tu as indexé un fichier, mais tu ne veux plus l'inclure au prochain commit.
3. Tu viens de créer un commit local, mais tu veux conserver les changements indexés pour le modifier.
4. Tu veux retirer le dernier commit tout en conservant les modifications dans les fichiers.
5. Un commit erroné a déjà été publié sur une branche partagée. Annule ses effets sans réécrire l'historique partagé.
6. Tu veux appliquer à la branche courante un commit précis provenant d'une autre branche.
7. Tu as perdu la trace d'un commit après une manipulation locale. Recherche les déplacements récents de HEAD.

8. Défi sécurité : explique pourquoi git reset --hard HEAD~1 est dangereux et dans quelles conditions il vaut mieux éviter de l'utiliser.

Commandes à travailler : git restore, git restore --staged, git reset --soft, git reset --mixed, git revert, git cherry-pick, git reflog.

Partie 6 — Stash, tags et historique

Exercice 14 — Mettre le travail de côté

1. Tu travailles sur le bouton « Liker », mais tu dois passer rapidement sur une autre branche sans créer de commit. Mets tes modifications de côté.
2. Affiche la liste des changements mis de côté.
3. Réapplique les changements et retire l'entrée de la liste si l'application réussit.
4. Quelle commande permet de réappliquer un stash sans le supprimer de la liste ?

Commandes à travailler : git stash push -m, git stash list, git stash pop, git stash apply.

Exercice 15 — Marquer une version

1. Crée un tag annoté v1.0.0.
2. Affiche les tags existants.
3. Affiche les détails du tag v1.0.0.
4. Explique pourquoi un tag est utile pour retrouver une version publiée.

Commandes à travailler : git tag -a, git tag, git show.

Partie 11 — Collaborer avec un dépôt distant

Exercice 16 — Configurer GitHub

1. Associe le dépôt local au dépôt distant avec le nom origin.
2. Affiche les URL du dépôt distant.
3. Publie la branche main et configure sa branche distante suivie.
4. Récupère les changements distants sans les fusionner automatiquement.
5. Affiche les branches locales et distantes connues.
6. Récupère et intègre les changements de develop.

7. Explique la différence entre git fetch et git pull.
8. Publie une branche de fonctionnalité.
9. Publie les tags locaux sur le dépôt distant.

Commandes à travailler : git remote add origin, git remote -v, git push -u origin main, git fetch origin, git branch -a, git pull origin develop, git push -u origin <branche>, git push origin --tags.

Exercice 17 — Gérer un désaccord entre versions

1. Un collègue a publié de nouveaux commits sur develop. Comment les récupérer ?
2. Ta branche locale contient aussi des commits non publiés. Comment intégrer les changements distants avec rebase ?
3. Que faire si cette intégration déclenche un conflit ?
4. Pourquoi faut-il être prudent avant de pousser une branche dont l'historique a été réécrit ?

Commandes à travailler : git fetch, git pull, git pull --rebase, git rebase --continue, git push.

Partie 12 — Pull Request et travail en équipe

Exercice 18 — Préparer une Pull Request

1. Crée feature/search-books à partir de develop.
2. Implémente une fonctionnalité de recherche de livres.
3. Vérifie les changements avant de les indexer.
4. Crée un commit clair.
5. Publie la branche distante.
6. Prépare une Pull Request vers develop.
7. Explique pourquoi il faut vérifier le diff et les tests avant la fusion.

Commandes à travailler : git switch -c, git status, git diff, git add, git diff --staged, git commit, git push -u origin.

Exercice 19 — Retirer un livre de la liste « À lire »

1. Crée feature/remove-from-read-list depuis develop.
2. Ajoute l'action Redux et le bouton correspondant.
3. Quel test manuel permet de vérifier que le livre est bien retiré ?
4. Comment intégrer cette fonctionnalité sans fusionner directement dans main ?

Partie 13 — Git Flow

Exercice 20 — Organiser le cycle de développement

1. Explique le rôle des branches main et develop.
2. Crée feature/like-book à partir de develop.
3. Décris comment intégrer la fonctionnalité terminée.
4. Crée release/1.0.0 à partir de develop.
5. Après validation, explique comment intégrer la release dans main et dans develop.
6. Crée un tag de version et publie-le.
7. Crée une branche hotfix/critical-fix pour corriger un problème urgent sur la version stable.
8. Explique comment réintégrer le correctif dans les branches concernées.

Commandes à travailler : git switch, git switch -c, git merge --no-ff, git tag -a, git push origin --tags.

Partie 14 — Grand défi final

Exercice 21 — Livrer Book Manager en équipe

1. Initialise le dépôt et configure son identité.
2. Crée l'application React avec Redux.
3. Configure .gitignore.
4. Crée le premier commit.
5. Développe la fonctionnalité « Liker » sur une branche dédiée.
6. Développe la liste « À lire » sur une autre branche.
7. Examine les modifications avec git diff et git diff --staged.
8. Fusionne les fonctionnalités dans develop.
9. Résous un conflit simulé.
10. Prépare une Pull Request.
11. Crée la release 1.0.0.
12. Crée et publie le tag.

QCM

Niveau 1 — Les bases de Git

Question 1 — Vérifier l'installation

Quelle commande affiche la version de Git installée ?

A. git check-version
B. git –version
C. git version –all
D. git status --version

Question 2 — Configurer le nom

Quelle commande configure globalement le nom de l'auteur des commits ?

A. git user.name "Nom"
B. git config user.name --local "Nom"
C. git config --global user.name "Nom"
D. git set name "Nom"

Question 3 — Afficher la configuration

Quelle commande affiche les paramètres de configuration Git ?

A. git config --list
B. git settings
C. git show-config
D. git status --config

Question 4 — Branche initiale

Comment définir main comme branche initiale par défaut pour les nouveaux dépôts ?

A. git branch --default main
B. git config --global init.defaultBranch main
C. git init main --default
D. git config branch.main true

Question 5 — Initialiser un dépôt

Que fait git init ?

A. Télécharge automatiquement un dépôt GitHub.
B. Crée obligatoirement un commit initial.
C. Initialise les métadonnées Git dans le dossier.
D. Publie le dossier sur un serveur.

Question 6 — Copier un dépôt

Quelle commande copie un dépôt distant existant avec son historique ?

A. git init

B. git clone URL
C. git fetch --new
D. git download

Question 7 — Vérifier l'état

Quelle commande distingue notamment les fichiers modifiés, indexés et non suivis ?

A. git status
B. git diff --all
C. git log
D. git branch

Question 8 — Comparer les modifications

Que montre normalement git diff sans option ?

A. Les différences entre le dépôt distant et local.
B. Les modifications non indexées par rapport à l'index.
C. La liste de tous les commits.
D. Les différences entre deux branches distantes uniquement.

Question 9 — Préparer un fichier

Quelle commande prépare README.md pour le prochain commit ?

A. git commit README.md
B. git prepare README.md
C. git add README.md
D. git stage-commit README.md

Question 10 — Vérifier l'index

Quelle commande affiche les différences déjà indexées ?

A. git diff --staged
B. git diff --working
C. git status --diff
D. git log --staged

Question 11 — Créer un commit

Quelle commande crée un commit avec un message ?

E. git save -m "Initial"
F. git commit -m "Initial"

G. git add -m "Initial"
H. git log -m "Initial"

Question 12 — Historique compact

Quelle commande affiche les commits en format compact ?

- A. git status --short-log
- B. git log --oneline
- C. git commit --list
- D. git branch --history

Question 13 — Graphe de l'historique

Quelle commande montre un graphe des branches et des commits connus ?

- A. git log --graph --decorate --all
- B. git diff --graph
- C. git status --all
- D. git show --branches

Question 14 — Examiner un commit

Que fait git show sans argument ?

- A. Supprime le dernier commit.
- B. Affiche généralement les détails du commit désigné par HEAD.
- C. Affiche uniquement les branches distantes.
- D. Réinitialise l'index.

Question 15 — Ignorer des fichiers

Quel fichier indique à Git quels fichiers non suivis ignorer ?

- A. .gitconfig
- B. git.ignore
- C. .gitignore
- D. ignore.git

Question 16 — Retirer un fichier du suivi

Un fichier .env est déjà suivi. Quelle commande le retire de l'index sans supprimer sa copie locale ?

- A. git rm --cached .env
- B. git restore .env
- C. git clean .env
- D. git delete --keep .env

Niveau 2 — Branches, fusion et rebase

Question 17 — Créer une branche

Quelle commande crée feature/search et bascule immédiatement dessus ?

- A. git branch feature/search --switch
- B. git switch -c feature/search
- C. git merge -c feature/search
- D. git init feature/search

Question 18 — Afficher les branches

Quelle commande affiche les branches locales et les références distantes connues ?

- A. git branch -a
- B. git branch -d
- C. git status -a
- D. git remote -b

Question 19 — Supprimer une branche fusionnée

Quelle commande supprime une branche locale déjà fusionnée ?

- A. git switch -d feature/search
- B. git rm feature/search
- C. git branch -d feature/search
- D. git delete branch feature/search

Question 20 — Changer de branche

Quand git switch main réussit, que se passe-t-il normalement ?

- A. Un nouveau commit est toujours créé.
- B. HEAD désigne la branche active main.
- C. Le dépôt distant est automatiquement mis à jour.
- D. Toutes les branches sont fusionnées.

Question 21 — Fusion sans fast-forward

À quoi sert git merge --no-ff feature/search ?

- A. À supprimer feature/search.
- B. À forcer un commit de fusion même si un fast-forward serait possible.
- C. À annuler le dernier commit.
- D. À rebaser automatiquement toutes les branches.

Question 22 — Résoudre un conflit

Après un conflit de fusion, quelle est la bonne démarche générale ?

- A. Ignorer le conflit et faire push.
- B. Supprimer le dépôt.
- C. Résoudre les fichiers, les ajouter avec git add, puis terminer la fusion.
- D. Exécuter toujours git reset --hard.

Question 23 — Récupérer les changements distants

Que fait git fetch origin ?

- A. Récupère les objets et références distants sans intégrer automatiquement les changements à la branche courante.
- B. Fusionne toujours origin/main dans main.
- C. Supprime automatiquement les branches locales obsolètes.
- D. Publie les commits locaux.

Question 24 — Comprendre le rebase

Quel est l'objectif principal de git rebase origin/main depuis une branche de fonctionnalité ?

- A. Supprimer main.
- B. Rejouer les commits locaux de la branche au-dessus de origin/main.
- C. Créer un tag automatiquement.
- D. Remplacer le répertoire de travail par une archive.

Question 25 — Continuer un rebase

Après avoir résolu un conflit pendant un rebase, quelle commande permet de poursuivre ?

- A. git merge --finish
- B. git rebase --continue
- C. git commit --rebase
- D. git fetch --continue

Question 26 — Annuler un rebase

Quelle commande abandonne le rebase en cours ?

- A. git rebase --abort
- B. git reset --continue
- C. git rebase --skip-all
- D. git restore --abort

Niveau 3 — Annulation et récupération

Question 27 — Abandonner des modifications non indexées

Tu veux abandonner les modifications non indexées de src/App.jsx. Quelle commande convient ?

- A. git restore src/App.jsx
- B. git restore --staged src/App.jsx
- C. git add src/App.jsx
- D. git commit src/App.jsx

Question 28 — Retirer un fichier de l'index

Tu veux retirer src/App.jsx de l'index mais conserver ses modifications dans le répertoire de travail. Que faire ?

- A. git restore src/App.jsx
- B. git restore --staged src/App.jsx
- C. git reset --hard
- D. git revert src/App.jsx

Question 29 — Reset soft

Quel est l'effet habituel de git reset --soft HEAD~1 ?

- A. Supprimer les changements du répertoire de travail.
- B. Déplacer la branche d'un commit en arrière en conservant les changements indexés.
- C. Créer un commit inverse.
- D. Envoyer le commit précédent sur le serveur.

Question 30 — Reset mixed

Quel est l'effet habituel de git reset --mixed HEAD~1 ?

- A. Déplacer la branche en arrière et réinitialiser l'index, tout en conservant les changements dans les fichiers.
- B. Tout supprimer du disque.
- C. Créer un tag.
- D. Créer un commit de fusion.

Question 31 — Reset hard

Pourquoi faut-il être prudent avec git reset --hard HEAD~1 ?

- A. Il publie automatiquement le dépôt.
- B. Il peut abandonner des changements du répertoire de travail et réinitialise l'index.
- C. Il ne fonctionne qu'avec GitHub.
- D. Il transforme le dépôt en dépôt distant.

Question 32 — Revert

Quel est l'avantage principal de git revert HEAD sur une branche partagée ?

- A. Il efface l'historique du commit ciblé.
- B. Il crée un nouveau commit qui inverse les changements.
- C. Il supprime automatiquement la branche distante.
- D. Il déplace HEAD sans créer de commit.

Question 33 — Cherry-pick

À quoi sert git cherry-pick <commit> ?

- A. À appliquer les changements d'un commit sélectionné à la branche courante.
- B. À renommer un dépôt distant.
- C. À supprimer tous les commits précédents.
- D. À créer une branche distante sans commit.

Question 34 — Reflog

À quoi sert git reflog ?

- A. À afficher les fichiers ignorés.
- B. À consulter l'historique local des déplacements de références, notamment HEAD.
- C. À publier les tags.
- D. À réinitialiser tous les dépôts distants.

Niveau 4 — Stash, tags et collaboration

Question 35 — Mettre le travail de côté

Quelle commande met de côté des modifications avec un message ?

- A. git stash push -m "Travail temporaire"
- B. git stash commit -m "Travail temporaire"
- C. git store -m "Travail temporaire"
- D. git reset --stash "Travail temporaire"

Question 36 — Réappliquer un stash

Quelle commande réapplique un stash sans le retirer de la liste ?

- A. git stash pop

- B. git stash apply
- C. git stash delete
- D. git stash remove

Question 37 — Comprendre stash pop

Quelle affirmation sur git stash pop est la plus exacte ?

- A. Il crée toujours un commit.
- B. Il tente d'appliquer le stash et le retire généralement de la liste si l'application réussit.
- C. Il supprime toutes les branches.
- D. Il envoie le stash au dépôt distant.

Question 38 — Créer un tag annoté

Quelle commande crée le tag annoté v1.0.0 ?

- A. git branch -a v1.0.0
- B. git tag -a v1.0.0 -m "Version 1.0.0"
- C. git commit --tag v1.0.0
- D. git release v1.0.0

Question 39 — Ajouter un dépôt distant

Quelle commande associe l'alias origin à une URL distante ?

- A. git remote add origin URL
- B. git branch origin URL
- C. git clone --remote origin URL
- D. git config origin URL

Question 40 — Afficher les URL distantes

Quelle commande affiche les URL des dépôts distants configurés ?

- A. git remote -v
- B. git branch -v
- C. git log -v

- D. git url --all

Question 41 — Publier une branche

Que fait généralement git push -u origin main ?

- A. Supprime main sur le serveur.
- B. Publie main et configure sa branche distante suivie.
- C. Fusionne origin/main dans main.
- D. Réinitialise HEAD.

Question 42 — Comprendre git pull

Quelle affirmation décrit le mieux git pull ?

- A. Il exécute toujours uniquement git status.
- B. Il récupère les changements distants et tente de les intégrer selon la stratégie configurée.
- C. Il ne fonctionne que si le dépôt n'a aucun commit.
- D. Il supprime toujours automatiquement les changements locaux.

Question 43 — Pull avec rebase

Que demande git pull --rebase ?

- A. De récupérer les changements puis rebaser les commits locaux non publiés au-dessus de la branche distante suivie.
- B. De supprimer la branche distante.
- C. De créer obligatoirement un commit de fusion.
- D. De réinitialiser le dépôt avec git init.

Question 44 — Publier les tags

Quelle commande envoie les tags locaux vers origin ?

- A. git tag --fetch origin
- B. git push origin --tags
- C. git remote add --tags
- D. git commit --tags

Niveau 5 — Git Flow, sécurité et situations réelles

Question 45 — Branche develop

Dans une organisation Git Flow classique, quel est le rôle général de develop ?

- A. Conserver uniquement les sauvegardes de fichiers.
- B. Servir de branche d'intégration des fonctionnalités en développement.
- C. Remplacer obligatoirement toutes les branches feature.
- D. Stocker uniquement les tags.

Question 46 — Branche release

À quoi sert généralement une branche release/1.0.0 ?

- A. À stabiliser et préparer une version avant sa publication.
- B. À supprimer l'historique de main.
- C. À remplacer le dépôt distant.
- D. À stocker les fichiers node_modules.

Question 47 — Branche hotfix

À partir de quelle branche crée-t-on généralement une branche hotfix dans Git Flow classique ?

- A. Depuis main, pour corriger la version publiée.
- B. Depuis une branche feature uniquement.
- C. Depuis un stash.
- D. Depuis .gitignore.

Question 48 — Pull Request

Quel est l'objectif d'une Pull Request ?

- A. Créer automatiquement un commit local sans revue.
- B. Proposer des changements pour revue et intégration dans une branche cible.
- C. Remplacer git add.
- D. Supprimer la nécessité de tester le code.

Question 49 — Les trois zones après git add

Tu exécutes git add App.jsx, puis tu modifies encore App.jsx sans refaire git add. Que contient l'index ?

- A. Toujours automatiquement la toute dernière modification.
- B. Le contenu ajouté lors du dernier git add, tandis que le répertoire de travail contient la nouvelle modification.
- C. Un nouveau commit.
- D. Obligatoirement la version du dépôt distant.

Question 50 — HEAD

Quelle action fait généralement avancer HEAD vers un nouveau commit ?

- A. git status
- B. git diff
- C. Un git commit réussi sur la branche active.
- D. git remote -v

Question 51 — Comparer l'index au commit

Tu veux comparer le contenu de l'index au dernier commit. Quelle commande utiliser ?

- A. git diff --staged
- B. git diff HEAD --cached-only-remote
- C. git log --diff
- D. git status --staged

Question 52 — Différence entre merge et rebase

Quelle affirmation est correcte ?

- A. Ils font exactement la même chose dans tous les cas.
- B. merge intègre des historiques, parfois avec un commit de fusion ; rebase rejoue des commits sur une nouvelle base.
- C. rebase ne change jamais les identifiants de commit.
- D. merge ne peut jamais provoquer de conflit.

Question 53 — Supprimer un fichier suivi

Que fait généralement git rm fichier sans --cached ?

- A. Retire le fichier du suivi et prépare sa suppression, généralement aussi dans le répertoire de travail.
- B. Ajoute le fichier à l'index.
- C. Crée un tag.
- D. Récupère le fichier depuis origin.

Question 54 — Commit erroné déjà publié

Un commit erroné est déjà publié sur une branche partagée. Quelle méthode est généralement la plus sûre pour annuler ses effets ?

- A. git reset --hard HEAD~1 puis forcer le push sans prévenir.
- B. git revert <commit>.
- C. Supprimer le dossier .git.
- D. git restore --staged .

Question 55 — Après git fetch

Après git fetch origin, quelle commande aide à consulter les branches distantes connues ?

- A. git branch -a
- B. git restore origin
- C. git stash list
- D. git config --global

Question 56 — Protéger un secret

Tu as un fichier .env contenant un secret. Quelle bonne pratique faut-il adopter ?

- A. Le committer pour garder une sauvegarde publique.
- B. L'ignorer avec .gitignore et vérifier qu'il n'est pas déjà suivi ; si un secret a été publié, le révoquer.
- C. Le renommer .env.txt et le publier.
- D. Le placer dans la Staging Area, mais jamais dans HEAD.

Question 57 — Comprendre reset --soft

Après git reset --soft HEAD~1, que devient généralement le contenu du commit retiré de la tête de branche ?

- A. Les changements restent indexés pour pouvoir être recommittés.
- B. Les fichiers sont systématiquement supprimés.
- C. Le commit est transformé en tag.
- D. Le dépôt distant est automatiquement modifié.

Question 58 — Identifiants après rebase

Pourquoi les identifiants de commits peuvent-ils changer après un rebase ?

- A. Git renomme seulement les branches.
- B. Les commits sont rejoués et de nouveaux objets commit sont créés.
- C. Git efface toujours les messages de commit.
- D. Les tags sont automatiquement convertis en branches.

Question 59 — Avant une Pull Request

Quelle vérification est la plus pertinente avant d'ouvrir une Pull Request ?

- A. Vérifier le diff, l'état du dépôt et les tests du projet.
- B. Exécuter git reset --hard dans tous les cas.
- C. Supprimer tous les tags.
- D. Ignorer les commits et publier directement main.

Question 60 — Cycle Git complet

Quel enchaînement représente le mieux le cycle courant pour enregistrer une modification ?

- A. git commit → git add → git diff → git init
- B. git status / git diff → git add → git diff --staged → git commit → git log
- C. git push → git init → git restore → git branch -d
- D. git tag → git clone → git reset --hard → git clean